



Mapeo objeto relacional.

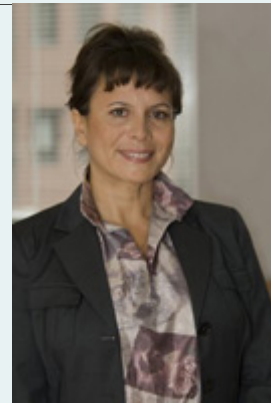


Caso práctico

En BK todo el desarrollo de software se hace siguiendo el paradigma de la Programación Orientada a Objetos. Sin embargo, la mayoría de las bases de datos con las que interaccionan sus aplicaciones son bases de datos relacionales, aunque también las hay orientadas a objetos.

Ada constata que parte de sus trabajadores se van a enfrentar con un dilema: ¿cómo usar objetos y clases para interactuar con bases de datos relacionales? Si en éstas sólo se pueden registrar tipos de datos simples, ¿cómo guardar ahí un objeto?

Este es el punto de partida del mapeo objeto-relacional, y los trabajadores de BK van a aprender cómo se trabaja en el sistema de almacenamiento de componentes propios de la Programación orientada a Objetos.



 [Ministerio de Educación \(Uso educativo nc\)](#)



  [Ministerio de Educación y Formación Profesional. \(Dominio público\)](#)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

 [Aviso Legal](#) 

1.- Concepto de Mapeo objeto-relacional.



Caso práctico



María percibe que sus aplicaciones, basadas en Programación Orientada a Objetos, no son almacenables de forma directa en los Sistemas Gestores de Bases de Datos que tienen en la empresa. ¿Cómo guardar un objeto en un sistema de almacenamiento basado en tablas?

Q Ministerio de Educación (Uso educativo nc)

A la hora de almacenar los datos de un programa orientado a objetos en una [base de datos relacional](#), surge un inconveniente debido a incompatibilidad de sistemas de [tipos de datos](#). En el software orientado a objetos, la información se representa como clases y objetos. En las bases de datos relacionales, como tablas y sus restricciones. Por tanto, para almacenar la información tratada en un programa orientado a objetos en una base de datos relacional es necesaria una traducción entre ambas formas.

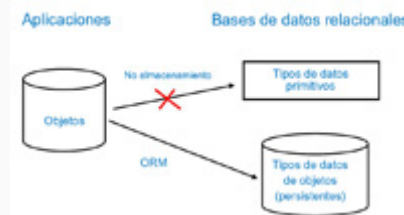
El mapeo objeto-relacional ([ORM](#)) soluciona este problema. Es una técnica de programación que se utiliza con el propósito de convertir datos entre el utilizado en un lenguaje de programación orientado a objetos y el utilizado en una base de datos relacional, gracias a la [persistencia](#). Esto posibilita el uso en las bases de datos relacionales de las características propias de la programación orientada a objetos (básicamente [herencia](#) y [polimorfismo](#)).

La mayoría de las aplicaciones se construyen usando técnicas de programación orientada a objetos; sin embargo, los sistemas de bases de datos más extendidos son de tipo relacional.

Las bases de datos más extendidas son del tipo relacional y estas sólo permiten guardar tipos de datos primitivos (enteros, cadenas de texto, etc.) por lo que no se puede guardar de forma directa los objetos de la aplicación en las tablas. Por tanto, se debe convertir los valores de los objetos en valores simples que puedan ser almacenados en una base de datos (y poder recuperarlos más tarde).

El mapeo objeto-relacional surge, pues, para dar respuesta a esta problemática: traducir los objetos a formas que puedan ser almacenadas en bases de datos

preservando las propiedades de los objetos y sus relaciones; estos objetos se dice entonces que son persistentes.



El ORM se encarga, de forma automática, de convertir los den [objetos](#) y [registros](#) y viceversa, simulando así tener una base de datos orientada a objetos.



Para saber más

En el siguiente enlace encontrarás un documento muy interesante sobre la problemática asociada al almacenamiento de objetos y al concepto de mapeo objeto-relacional.

 [Introducción al mapeo objeto-relacional !\[\]\(2f36c159ea3670f7a62f64a4f1cf5c05_img.jpg\) \(https://informaticaviernes2011.files.wordpress.com/2011/07/or_m_victor_raquel_berni.pdf\)](https://informaticaviernes2011.files.wordpress.com/2011/07/or_m_victor_raquel_berni.pdf) (0.43 MB)

2.- Herramientas ORM. Características y herramientas más utilizadas.



Caso práctico

En la empresa se ponen manos a la obra para estudiar las herramientas que permitan almacenar objetos y clases en bases de datos relacionales.

¿Qué son y qué características tienen las herramientas ORM?



 [Ministerio de Educación \(Uso educativo nc\)](#)

Ya hemos dicho que las herramientas ORM se utilizan para dar solución al problema de que, en la programación orientada a objetos, la gestión de datos se implementa usando objetos; sin embargo, en los sistemas de gestión de bases de datos SQL sólo se pueden almacenar y manipular valores escalares organizados en tablas relacionales.

Object Relational Mapping (ORM) es la herramienta que nos sirve para transformar representaciones de datos de los Sistemas de Bases de Datos Relacionales, a representaciones (Modelos) de objetos. Dado a que los RDBMS carecen de la flexibilidad para representar datos no escalares, la existencia de un ORM es fundamental para el desarrollo de sistemas de software robustos y escalables.

En el modelo relacional, cada fila en la tabla se mapea a un objeto y cada columna a una propiedad.

Las herramientas ORM pues, actúan como un puente que conecta las ventajas de los RDBMS con la buena representación de estos en un lenguaje Orientado a Objetos, o, dicho en otras palabras, nos lleva de la base de datos al lenguaje de programación.



Autoevaluación

Utilizando una herramienta ORM podemos recuperar los objetos de un programa en formas compatibles con el sistema de almacenamiento de los

gestores de bases de datos relacionales.

- ☐ No.
- ☐ Si.

Incorrecta. Realmente, esa es la función principal de una herramienta ORM.

Correcto. Efectivamente, y esos objetos se llaman persistentes.

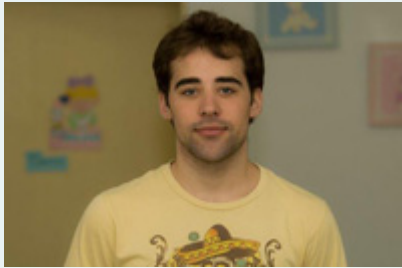
Solución

1. Incorrecto (#answer-13_63)
2. Opción correcta (#answer-13_82)

2.1.- Características.



Caso práctico



Q Ministerio de Educación (Uso educativo nc)

Antonio no sabe exactamente qué tiene que utilizar ni qué ventajas obtendrá con el uso de las herramientas de mapeo.

¿Cuáles son las características de las herramientas ORM?

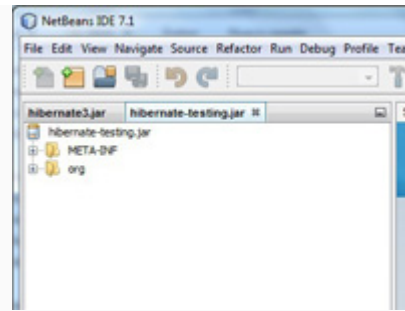
Las herramientas ORM facilitan el mapeo de atributos entre una base de datos relacional y el modelo de objetos de una aplicación, la forma de hacerlo es usando *Hibernate* de forma nativa, mediante archivos declarativos ([XML](#)) o haciendo uso de *Jakarta Persistence* y anotaciones dentro del código de las clases que forman parte del dominio, ambas estrategias nos van a permitir establecer estas relaciones. Gracias a las ORM, podemos conectar con una base de datos relacional para extraer la información contenida en objetos de programas que están almacenados. Para ello, sólo tendremos que definir la forma en la que establecer la correspondencia entre las clases y las tablas una sola vez (indicando qué propiedad se corresponde con cada columna, qué clase con cada tabla, etc.). Una vez hecho esto, podremos utilizar [POJOs](#) de nuestra aplicación e indicar a la *ORM* que los haga persistentes, consiguiendo que una sola herramienta pueda leer o escribir en la base de datos utilizando [VOs](#) directamente.

Una herramienta ORM permite tomar un objeto Java y hacerlo persistente, carga el objeto de la base de datos a memoria y permite hacer consultas a las tablas de la base de datos.

Ventajas de ORM.

- Ayudan a reducir el tiempo de desarrollo de software. La mayoría de las herramientas ORM disponibles, permiten la creación del modelo a través del esquema de la base de datos, es decir, el usuario crea la base de datos y la herramienta automáticamente lee el esquema de tablas y relaciones y crea un modelo ajustado.
- Abstracción de la base de datos.

- Reutilización.
- Permiten persistir objetos a través de un método `entityManager.persist()` y generar el SQL correspondiente.
- Permiten recuperar los objetos persistidos a través de un método `entityManager.getReference()`.
- Lenguaje propio para realizar las consultas.
- Independencia de la base de datos.
- Incentivan la [portabilidad](#) y [escalabilidad](#) de los programas de software.



Q Verónica Cabrerizo (Captura de pantalla de NetBeans) ([GNU/GPL](#))

Desventajas de ORM.

- Tiempo utilizado en el aprendizaje. Este tipo de herramientas suelen ser complejas por lo que su correcta utilización lleva un tiempo que hay que emplear en ver el funcionamiento correcto y ver todo el partido que se le puede sacar.
 - Menor rendimiento (aplicaciones algo más lentas). Esto es debido a que todas las consultas que se hagan sobre la base de datos, el sistema primero deberá de transformarlas al lenguaje propio de la herramienta, luego leer los registros y por último crear los objetos.
 - Sistemas complejos. Normalmente la utilidad de ORM desciende con la mayor complejidad del sistema relacional.
-

2.2.- Herramientas ORM más utilizadas.



Caso práctico

Sabiendo ya la función de las herramientas ORM, el siguiente punto es el análisis de las ORM existentes en el mercado, junto a sus características más destacables, para decidir cuál es la que mejor se adapta a nuestras necesidades.



🔍 [Ministerio de Educación \(Uso educativo nc\)](#)

Entre las herramientas *ORM* más relevantes encontramos las siguientes:

Hibernate:

Hibernate es una herramienta de Mapeo objeto-relacional (ORM) para la plataforma Java (y disponible también para .Net con el nombre de NHibernate) que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación. Utiliza archivos declarativos (XML) o anotaciones en los *beans* de las entidades que permiten establecer estas relaciones.

Hibernate es software libre, distribuido bajo los términos de la licencia GNU LGPL.



🔍 [hibernate.org](#) (GNU/GPL)

Java Persistence Api (ahora Jakarta Persistence):

El Java Persistence API (JPA) es una especificación de Sun Microsystems para la persistencia de objetos Java a cualquier base de datos relacional. Esta API fue desarrollada para la plataforma JEE e incluida en el estándar de EJB 3.0, formando parte de la Java Specification Request JSR 220.

Para su utilización, JPA requiere de J2SE 1.5 (también conocida como Java 5) o superior, ya que hace uso intensivo de las nuevas características de lenguaje Java, como las anotaciones y los genéricos.

iBatis:

iBatis es un framework de persistencia desarrollado por la Apache software Foundation. Al igual que el resto de los proyectos desarrollados por la ASF,

iBatis es una herramienta de código libre.

iBatis sigue el mismo esquema de uso que Hibernate; se apoya en ficheros de mapeo XML para persistir la información contenida en los objetos en un [repositorio](#) relacional.



Q ibatis.apache.org (Copyright (Cita))

En el siguiente documento encontrarás las características más destacables de la herramienta Hibernate.

[Aspectos de la configuración de Hibernate](#) (0.19 MB)



Autoevaluación

La herramienta ORM Hibernate se caracteriza por:

- ☐ No ser compatible con la plataforma .Net.
- ☐ Ser software libre.
- ☐ Ser una especificación de Sun Microsystems.

Incorrecta, existe la variante NHibernate para este framework.

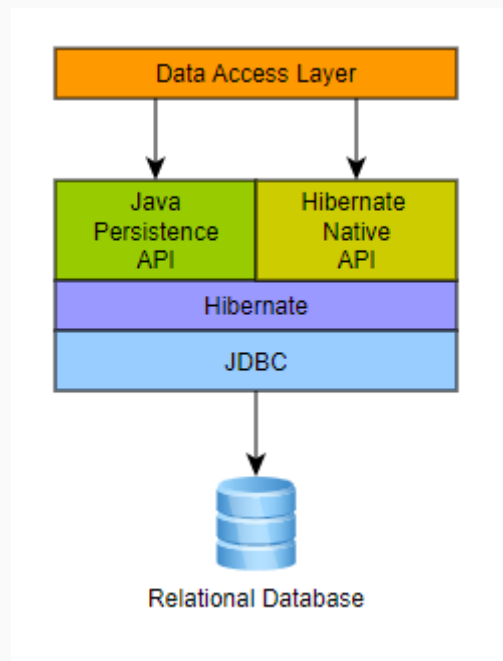
Correcto. Efectivamente, tiene licencia GNU LGPL.

Incorrecta. En este caso, se está refiriendo a JPA.

Solución

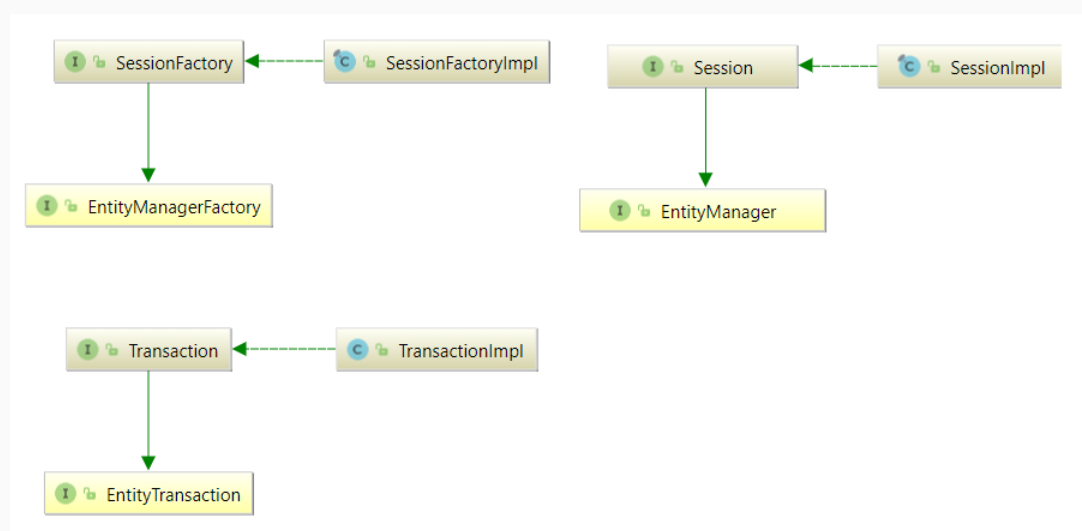
1. Incorrecto (#answer-20_63)
2. Opción correcta (#answer-20_91)
3. Incorrecto (#answer-20_94)

3.- Arquitectura de Hibernate



Hibernate, es una solución *ORM*, efectiva, como se ha comentado en puntos anteriores, se ubica entre la capa de acceso a datos de la aplicación Java y la base de datos relacional, como se puede ver en el diagrama anterior. La aplicación Java hace uso de las *API* de *Hibernate* para cargar, almacenar, consultar, etc. sus datos de dominio. Aquí presentaremos las *API* esenciales de *Hibernate* a modo de introducción.

Como proveedor de persistencia de *Jakarta*, *Hibernate* implementa las especificaciones de la *API* de persistencia de Java y la asociación entre las interfaces de persistencia de *Jakarta* y las implementaciones específicas de *Hibernate* se pueden visualizar en el siguiente diagrama:



Fábrica de sesiones (org.hibernate.SessionFactory)

Es una representación segura para subprocessos (e inmutable) de la asignación del modelo de dominio de la aplicación a una base de datos. Actúa como una fábrica de *org.hibernate.Session* instancias. El *EntityManagerFactory* es el equivalente de Persistencia de *Jakarta* a la clase *SessionFactory* y, básicamente,

esos dos convergen en la misma implementación *SessionFactory*.

Un objeto de tipo *SessionFactory* es muy costoso de crear, por lo que, para cualquier base de datos dada, la aplicación debe tener solo un objeto de este tipo. Cuando se instancia de tipo *SesiónFactory*, se mantiene los servicios que *Hibernate* usa durante toda la sesión, como cachés de segundo nivel, grupos de conexiones, integraciones de sistemas de transacciones, etc.

Sesión (*org.hibernate.Session*)

Se trata de un objeto de corta duración de un solo subproceso que modela conceptualmente una "*Unidad de trabajo*". En la nomenclatura de Persistencia de Jakarta, el *Session* está representado por un *EntityManager*.

En segundo plano , los objetos de tipo *Session* de *Hibernate* envuelven un *JDBC java.sql.Connection* y actúa como una fábrica de instancias de tipo *org.hibernate.Transaction*. Manteniendo un contexto de persistencia generalmente de "lectura repetible" (caché de primer nivel) del modelo de dominio de la aplicación.

Transacción (*org.hibernate.Transaction*)

Se trata de un objeto de un solo subproceso y de corta duración utilizado por la aplicación para demarcar los límites de transacciones físicas individuales. *EntityTransaction* es el equivalente de Persistencia de Jakarta y ambos actúan como una API de abstracción para aislar la aplicación del sistema de transacciones subyacente en uso (*JDBC* o *JTA*).



Para saber más

Para saber más sobre Hibernate y su arquitectura visita el siguiente

 [enlace](#) .